Lightweight Cryptography Benchmarking using FELICS Framework

Your Name Here

Edinburgh Napier University

Abstract

This paper includes a literature review of lightweight encryption and evaluates the performance of lightweight ciphers using the FELICS benchmarking framework. The SKINNY lightweight block cipher is compared against AES, SPECK, and SIMON.

Keywords: Lightweight Cryptography, FELICS, SKINNY, Encryption, Benchmarking

1. Introduction

Lightweight cryptography is increasingly important as the use of Internet-connected devices continues to grow. Proposed legislation mandating minimum security standards attempts to close the gaps on these threats, but hostile actors continue to challenge existing protections through a variety of attacks designed to compromise the security of devices at all levels of IoT architecture [**?**].

2. Literature Review

2.1. ISO/IEC 29192 – Lightweight Cryptography Standards

The ISO/IEC 29192 standard defines the minimum security requirements and performance considerations for lightweight cryptographic systems, ensuring interoperability and sufficient protection for constrained environments.

2.2. SKINNY Lightweight Encryption

SKINNY is a lightweight block cipher designed to achieve security comparable to AES while being significantly more efficient in hardware and software. SKINNY supports various block and key sizes, and it targets IoT and embedded applications.

3. Tools

The Fair Evaluation of Lightweight Cryptographic Systems (FELICS) benchmarking framework was created by CryptoLux. CryptoLux are a research group within the University of Luxembourg.

FELICS was developed to introduce a fair way of testing encryption algorithms by providing a standardised baseline across different platforms. Existing research often uses inconsistent environments, making comparisons unreliable.

Definitions from FELICS:

1. Key sizes and Block sizes are in bits [b].
2. Code size (Code) and RAM allocation (RAM) are in bytes [B].
3. Code execution time (Time) is measured in CPU cycles [cyc.].
4. Security Level (Sec.) is the ratio of rounds broken to total rounds.
5. If no attack is successful, the security level is set to -1.
6. Assembly implementations are italicised in results.
7. Cipher-r indicates the cipher with 'r' rounds instead of the default.

4. Metrics

FELICS allows benchmarking of:

1. RAM overheads
2. Size on disk
3. Cipher execution time

5. Test Plan

This testing focuses only on block ciphers. SKINNY-128-128 is compared to AES-128-128 due to similar key sizes. Speck-64-96 and Simon-64-96 are also benchmarked to highlight how SKINNY performs against other lightweight ciphers it was designed to compete with.

6. Implementation

6.1. Encryption Key Schedule

The following code excerpt shows part of the key scheduling process for SKINNY-128:

```
#include <stdint.h>

#include "constants.h"


/*
 *
 * Cipher constants
 *
 */
/* Replace with the cipher constants definition */
SBOX_BYTE SBOX[256] = {
        0x65 , 0x4c , 0x6a , 0x42 , 0x4b , 0x63 , 0x43 , 0x6b ,
            ↪  0x55 , 0x75 , 0x5a , 0x7a , 0x53 , 0x73 , 0x5b ,
            ↪  0x7b ,
        0x35 , 0x8c , 0x3a , 0x81 , 0x89 , 0x33 , 0x80 , 0x3b ,
            ↪  0x95 , 0x25 , 0x98 , 0x2a , 0x90 , 0x23 , 0x99 ,
            ↪  0x2b ,
        0xe5 , 0xcc , 0xe8 , 0xc1 , 0xc9 , 0xe0 , 0xc0 , 0xe9 ,
            ↪  0xd5 , 0xf5 , 0xd8 , 0xf8 , 0xd0 , 0xf0 , 0xd9 ,
            ↪  0xf9 ,
        0xa5 , 0x1c , 0xa8 , 0x12 , 0x1b , 0xa0 , 0x13 , 0xa9 ,
            ↪  0x05 , 0xb5 , 0x0a , 0xb8 , 0x03 , 0xb0 , 0x0b ,
            ↪  0xb9 ,
        0x32 , 0x88 , 0x3c , 0x85 , 0x8d , 0x34 , 0x84 , 0x3d ,
            ↪  0x91 , 0x22 , 0x9c , 0x2c , 0x94 , 0x24 , 0x9d ,
            ↪  0x2d ,
        0x62 , 0x4a , 0x6c , 0x45 , 0x4d , 0x64 , 0x44 , 0x6d ,
            ↪  0x52 , 0x72 , 0x5c , 0x7c , 0x54 , 0x74 , 0x5d ,
            ↪  0x7d ,
        0xa1 , 0x1a , 0xac , 0x15 , 0x1d , 0xa4 , 0x14 , 0xad ,
            ↪  0x02 , 0xb1 , 0x0c , 0xbc , 0x04 , 0xb4 , 0x0d ,
```

```
        ↪ 0xbd ,
    0xe1 , 0xc8 , 0xec , 0xc5 , 0xcd , 0xe4 , 0xc4 , 0xed ,
        ↪ 0xd1 , 0xf1 , 0xdc , 0xfc , 0xd4 , 0xf4 , 0xdd ,
        ↪ 0xfd ,
    0x36 , 0x8e , 0x38 , 0x82 , 0x8b , 0x30 , 0x83 , 0x39 ,
        ↪ 0x96 , 0x26 , 0x9a , 0x28 , 0x93 , 0x20 , 0x9b ,
        ↪ 0x29 ,
    0x66 , 0x4e , 0x68 , 0x41 , 0x49 , 0x60 , 0x40 , 0x69 ,
        ↪ 0x56 , 0x76 , 0x58 , 0x78 , 0x50 , 0x70 , 0x59 ,
        ↪ 0x79 ,
    0xa6 , 0x1e , 0xaa , 0x11 , 0x19 , 0xa3 , 0x10 , 0xab ,
        ↪ 0x06 , 0xb6 , 0x08 , 0xba , 0x00 , 0xb3 , 0x09 ,
        ↪ 0xbb ,
    0xe6 , 0xce , 0xea , 0xc2 , 0xcb , 0xe3 , 0xc3 , 0xeb ,
        ↪ 0xd6 , 0xf6 , 0xda , 0xfa , 0xd3 , 0xf3 , 0xdb ,
        ↪ 0xfb ,
    0x31 , 0x8a , 0x3e , 0x86 , 0x8f , 0x37 , 0x87 , 0x3f ,
        ↪ 0x92 , 0x21 , 0x9e , 0x2e , 0x97 , 0x27 , 0x9f ,
        ↪ 0x2f ,
    0x61 , 0x48 , 0x6e , 0x46 , 0x4f , 0x67 , 0x47 , 0x6f ,
        ↪ 0x51 , 0x71 , 0x5e , 0x7e , 0x57 , 0x77 , 0x5f ,
        ↪ 0x7f ,
    0xa2 , 0x18 , 0xae , 0x16 , 0x1f , 0xa7 , 0x17 , 0xaf ,
        ↪ 0x01 , 0xb2 , 0x0e , 0xbe , 0x07 , 0xb7 , 0x0f ,
        ↪ 0xbf ,
    0xe2 , 0xca , 0xee , 0xc6 , 0xcf , 0xe7 , 0xc7 , 0xef ,
        ↪ 0xd2 , 0xf2 , 0xde , 0xfe , 0xd7 , 0xf7 , 0xdf ,
        ↪ 0xff};

SBOX_BYTE INV_SBOX[256] = {
    0xac , 0xe8 , 0x68 , 0x3c , 0x6c , 0x38 , 0xa8 , 0xec ,
        ↪ 0xaa , 0xae , 0x3a , 0x3e , 0x6a , 0x6e , 0xea ,
        ↪ 0xee ,
    0xa6 , 0xa3 , 0x33 , 0x36 , 0x66 , 0x63 , 0xe3 , 0xe6 ,
        ↪ 0xe1 , 0xa4 , 0x61 , 0x34 , 0x31 , 0x64 , 0xa1 ,
        ↪ 0xe4 ,
    0x8d , 0xc9 , 0x49 , 0x1d , 0x4d , 0x19 , 0x89 , 0xcd ,
        ↪ 0x8b , 0x8f , 0x1b , 0x1f , 0x4b , 0x4f , 0xcb ,
```

```
    ↪ 0xcf ,
0x85 , 0xc0 , 0x40 , 0x15 , 0x45 , 0x10 , 0x80 , 0xc5 ,
    ↪ 0x82 , 0x87 , 0x12 , 0x17 , 0x42 , 0x47 , 0xc2 ,
    ↪ 0xc7 ,
0x96 , 0x93 , 0x03 , 0x06 , 0x56 , 0x53 , 0xd3 , 0xd6 ,
    ↪ 0xd1 , 0x94 , 0x51 , 0x04 , 0x01 , 0x54 , 0x91 ,
    ↪ 0xd4 ,
0x9c , 0xd8 , 0x58 , 0x0c , 0x5c , 0x08 , 0x98 , 0xdc ,
    ↪ 0x9a , 0x9e , 0x0a , 0x0e , 0x5a , 0x5e , 0xda ,
    ↪ 0xde ,
0x95 , 0xd0 , 0x50 , 0x05 , 0x55 , 0x00 , 0x90 , 0xd5 ,
    ↪ 0x92 , 0x97 , 0x02 , 0x07 , 0x52 , 0x57 , 0xd2 ,
    ↪ 0xd7 ,
0x9d , 0xd9 , 0x59 , 0x0d , 0x5d , 0x09 , 0x99 , 0xdd ,
    ↪ 0x9b , 0x9f , 0x0b , 0x0f , 0x5b , 0x5f , 0xdb ,
    ↪ 0xdf ,
0x16 , 0x13 , 0x83 , 0x86 , 0x46 , 0x43 , 0xc3 , 0xc6 ,
    ↪ 0x41 , 0x14 , 0xc1 , 0x84 , 0x11 , 0x44 , 0x81 ,
    ↪ 0xc4 ,
0x1c , 0x48 , 0xc8 , 0x8c , 0x4c , 0x18 , 0x88 , 0xcc ,
    ↪ 0x1a , 0x1e , 0x8a , 0x8e , 0x4a , 0x4e , 0xca ,
    ↪ 0xce ,
0x35 , 0x60 , 0xe0 , 0xa5 , 0x65 , 0x30 , 0xa0 , 0xe5 ,
    ↪ 0x32 , 0x37 , 0xa2 , 0xa7 , 0x62 , 0x67 , 0xe2 ,
    ↪ 0xe7 ,
0x3d , 0x69 , 0xe9 , 0xad , 0x6d , 0x39 , 0xa9 , 0xed ,
    ↪ 0x3b , 0x3f , 0xab , 0xaf , 0x6b , 0x6f , 0xeb ,
    ↪ 0xef ,
0x26 , 0x23 , 0xb3 , 0xb6 , 0x76 , 0x73 , 0xf3 , 0xf6 ,
    ↪ 0x71 , 0x24 , 0xf1 , 0xb4 , 0x21 , 0x74 , 0xb1 ,
    ↪ 0xf4 ,
0x2c , 0x78 , 0xf8 , 0xbc , 0x7c , 0x28 , 0xb8 , 0xfc ,
    ↪ 0x2a , 0x2e , 0xba , 0xbe , 0x7a , 0x7e , 0xfa ,
    ↪ 0xfe ,
0x25 , 0x70 , 0xf0 , 0xb5 , 0x75 , 0x20 , 0xb0 , 0xf5 ,
    ↪ 0x22 , 0x27 , 0xb2 , 0xb7 , 0x72 , 0x77 , 0xf2 ,
    ↪ 0xf7 ,
0x2d , 0x79 , 0xf9 , 0xbd , 0x7d , 0x29 , 0xb9 , 0xfd ,
```

```
            ↪  0x2b , 0x2f , 0xbb , 0xbf , 0x7b , 0x7f , 0xfb ,
            ↪  0xff};


ROUND_CONST_BYTE RC[40] = {
        0x01, 0x03, 0x07, 0x0F, 0x1F, 0x3E, 0x3D, 0x3B, 0x37, 0
            ↪ x2F,
        0x1E, 0x3C, 0x39, 0x33, 0x27, 0x0E, 0x1D, 0x3A, 0x35, 0
            ↪ x2B,
        0x16, 0x2C, 0x18, 0x30, 0x21, 0x02, 0x05, 0x0B, 0x17, 0
            ↪ x2E,
        0x1C, 0x38, 0x31, 0x23, 0x06, 0x0D, 0x1B, 0x36, 0x2D, 0
            ↪ x1A,};
```

6.2. FELICS Benchmarking Results

6.3. Size on Disk

Code size

SKINNY128-128

Component	ROM	text	data	bss	dec
cipher	2382	2110	272	4	2386
common	2010	2010	0	0	2010
constants	552	40	512	0	552
decryption_key_schedule	46	46	0	0	46
decrypt	46	46	0	0	46
encryption_key_schedule	46	46	0	0	46
encrypt	46	46	0	0	46
main	262	262	0	0	262
test_vectors	48	48	0	0	48

AES-128-128

Component	ROM	text	data	bss	dec
cipher	7490	7218	272	4	7494
common	2010	2010	0	0	2010
decryption_key_schedule	46	46	0	0	46
decrypt	2193	2193	0	0	2193
encryption_key_schedule	306	306	0	0	306
encrypt	2198	2198	0	0	2198
gmul_o	325	325	0	0	325
inv_sbox	256	256	0	0	256
main	262	262	0	0	262
rc_tab	10	10	0	0	10
sbox	256	256	0	0	256
test_vectors	48	48	0	0	48

Speck-64-94

Component	ROM	text	data	bss	dec
cipher	2566	2294	272	4	2570
common	593	593	0	0	593
decryption_key_schedule	46	46	0	0	46
decrypt	114	114	0	0	114
encryption_key_schedule	154	154	0	0	154
encrypt	113	113	0	0	113
main	262	262	0	0	262
test_vectors	28	28	0	0	28

Simon-64-96

Component	ROM	text	data	bss	dec
cipher	2976	2704	272	4	2980
common	593	593	0	0	593
constants	42	42	0	0	42
decryption_key_schedule	46	46	0	0	46
decrypt	192	192	0	0	192
encryption_key_schedule	324	324	0	0	324
encrypt	192	192	0	0	192
main	262	262	0	0	262
test_vectors	28	28	0	0	28

Figure 1: Size on Disk Comparison

6.4. Code Execution Time

SKINNY-128-128

Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
0	3897	4233	0	3770

AES128-128

Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
0	4594	10268	0	13173

Speck-64-96

Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
0	4386	4640	0	4160

Simon-64-96

Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
0	4499	4757	0	4195

Figure 2: Cipher Execution Time Comparison

SKINNY proved to be faster than AES, SPECK, and SIMON in terms of code execution time. This result is expected, as SKINNY was designed with efficiency and lightweight operations in mind.

6.5. RAM Usage

Skinny-128-128

Block Size	Key Size	R.K.s Size	Data RAM	Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
16	16	320	352	4	0	0	0	0

AES-128-128

Block Size	Key Size	R.K.s Size	Data RAM	Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
16	16	176	208	160	17	124	0	156

Speck-64-96

Block Size	Key Size	R.K.s Size	Data RAM	Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
8	12	104	124	20	16	8	0	8

Simon-64-96

Block Size	Key Size	R.K.s Size	Data RAM	Scenario	Enc. K.S.	Enc.	Dec. K.S.	Dec.
8	12	168	188	24	20	17	0	17

Figure 3: RAM Allocation Comparison

While SKINNY outperformed others in execution speed, it was found to use **more RAM** compared to AES, SPECK, and SIMON. This could be due to SKINNY's 16-bit internal block size structure requiring additional memory overhead.

7. Evaluation

The experiments demonstrated that SKINNY has better execution time performance than AES, SPECK, and SIMON. However, the RAM consumption is higher, indicating that for extremely memory-constrained environments, SKINNY might not always be the best choice despite its speed advantage.

8. Future Research

Future work could involve testing these lightweight algorithms directly on physical IoT devices as proposed in [**?**]. Hardware-based encryption testing would provide more realistic insights into energy consumption, latency, and long-term device reliability.

[**?**]

9. Conclusion

This study included a review of lightweight cryptography trends and evaluated several lightweight ciphers using the FELICS framework. SKINNY was found to perform better in terms of speed but had higher RAM usage, highlighting the common trade-offs between performance and resource consumption in cryptographic algorithm design.